Recursive Think-Answer Process for LLMs and VLMs

Byung-Kwan Lee* †
KAIST
leebk@kaist.ac.kr

Youngchae Chee*
KAIST
litcoderr@kaist.ac.kr

Yong Man Ro
KAIST
ymro@kaist.ac.kr

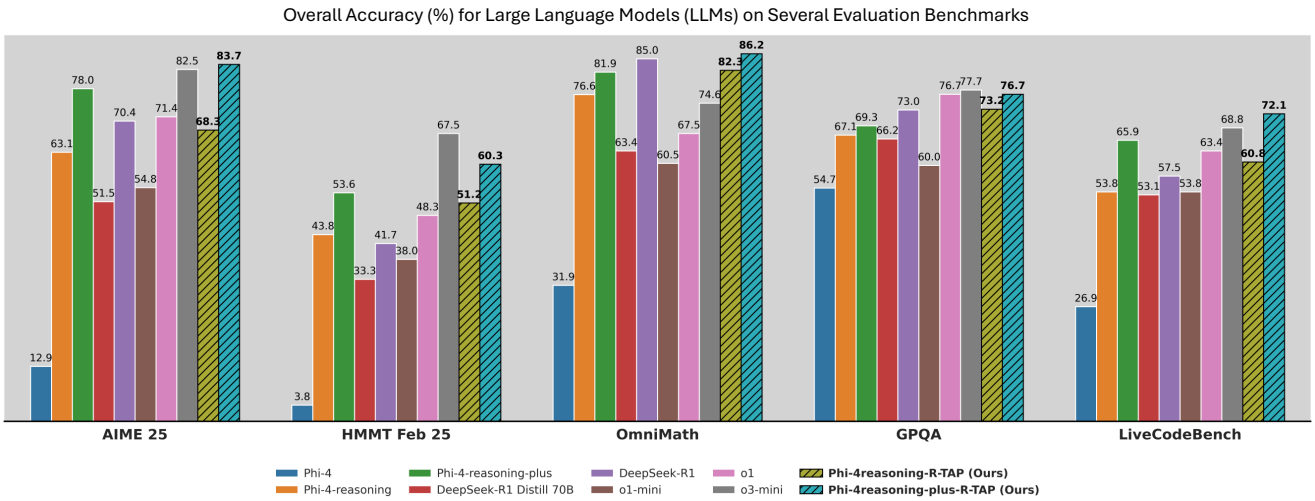


Figure 1. Overall accuracy (%) of numerous large language models (LLMs) on five evaluation benchmarks—AIME25 [65], HMMT Feb 25 [2], OmniMath [29], GPQA [25], and LiveCodeBench [43].

Abstract

Think–Answer reasoners such as DeepSeek-R1 have made notable progress by leveraging interpretable internal reasoning. However, despite the frequent presence of self-reflective cues like "Oops!", they remain vulnerable to output errors during single-pass inference. To address this limitation, we propose an efficient Recursive Think–Answer Process (R-TAP) that enables models to engage in iterative reasoning cycles and generate more accurate answers, going beyond conventional single-pass approaches. Central to this approach is a confidence generator that evaluates the certainty of model responses and guides subsequent improvements. By incorporating two complementary rewards—Recursively Confidence Increase Reward and Final Answer Confidence Reward—we show that R-TAP-enhanced models consistently outperform conventional single-pass methods for both large language models (LLMs) and vision-language models (VLMs). Moreover, by analyzing the frequency of "Oops"-like expressions in model responses, we find that R-TAP–applied models exhibit significantly fewer self-reflective patterns, resulting in more stable and faster inference-time reasoning. We hope R-TAP pave the way evolving into efficient and elaborated methods to refine the reasoning processes of future AI.

1. Introduction

Recent advances in a Think–Answer process-equipped models—such as OpenAI's o1 [42] and DeepSeek-R1 [35]—have demonstrated that explicitly separating the reasoning and answering stages can substantially enhance problem-solving performance. By following a Think–Answer scheme, these models surpass direct-prediction approaches and achieve remarkable capabilities in domains such as mathematical reasoning and competitive programming. This paradigm has also been extended to multimodal settings, where recent VLMs leverage a Think-

*Equal contribution
†Currently Research Scientist at NVIDIA

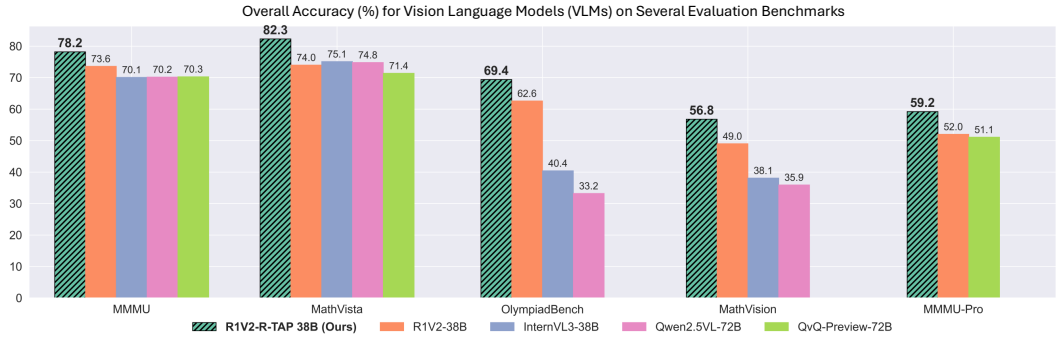


Figure 2. Overall accuracy (%) of numerous vision language models (VLMs) on five evaluation benchmarks—MMMU [94], MathVista [58], OlympiadBench [36], MathVision [78], and MMMU-Pro [95].

SYSTEM: You are a helpful AI Assistant that provides well-reasoned and detailed responses. You first think about the reasoning process as an internal monologue and then provide the user with the answer. Respond in the following format: <think> </think>\n<answer> </answer>

USER: Albert writes 2025 numbers a_{1}, \ldots, a_{2025} in a circle on a blackboard. Initially, each of the numbers is uniformly and independently sampled at random from the interval [0,1]. Then, each second, he \emph{simultaneously} replaces a_{i} with \max \left(a_{i-1}, a_{i}, a_{i+1}\right) for all i=1,2, \ldots, 2025 (where a_{0}=a_{2025} and a_{2026}=a_{1}). Compute the expected value of the number of distinct values remaining after 100 seconds.

ASSISTANT: <think> We want the expected number of distinct values remaining after 100 synchronous "max-with-neighbors" updates on a circle of length 2025. After \(t\) steps, each final value is the maximum over some interval of the initial samples. A natural first guess is that the "influence window" of each initial point expands one step per second in both directions, giving a window length of \(2t+1\). If these windows partition the circle roughly evenly, one might expect about \[\frac{2025}{2\cdot100 + 1} ~=~ \frac{2025}{201} \] distinct values, since each block of length 201 contributes one surviving maximum on average. </think>
<answer> \frac{2025}{201} </answer>

<think> **Ah—on reflection**, I realize that because we update simultaneously, the propagation of a crest might not spread symmetrically in both directions in exactly the same way I assumed. Instead, each step seems to effectively pull maxima inwards by one, but maybe only one-sided propagation of length 100 matters, giving window length 100 rather than 201. If the relevant block size is really \(t\) (not \(2t+1\)), then we'd predict \[\frac{2025}{100} \] distinct values remaining. </think>
<answer> \frac{2025}{100} </answer>

<think> **I made another off-by-one error.** In fact, each initial peak influences its neighbors one step per second on each side, but since we include the center itself at time 0, the true window length after \(t\) seconds is \(t+1\) in each direction—yet because it's circular, one only counts distinct windows of length \(t+1\) before they overlap. The well-known result for this max–smoothing process on a circle is that one ends up with about \(n/(t+1)\) survivors. Here \(n=2025\) and \(t=100\), so the correct expected number is \[\frac{2025}{100+1} = \frac{2025}{101}. \] </think>
<answer> \frac{2025}{101} </answer>

Figure 3. Qualitative example of recursive think–answer process on a combinatorics question. The model iteratively refines its solution across multiple reasoning cycles, successfully correcting initial misconceptions such as off-by-one errors.

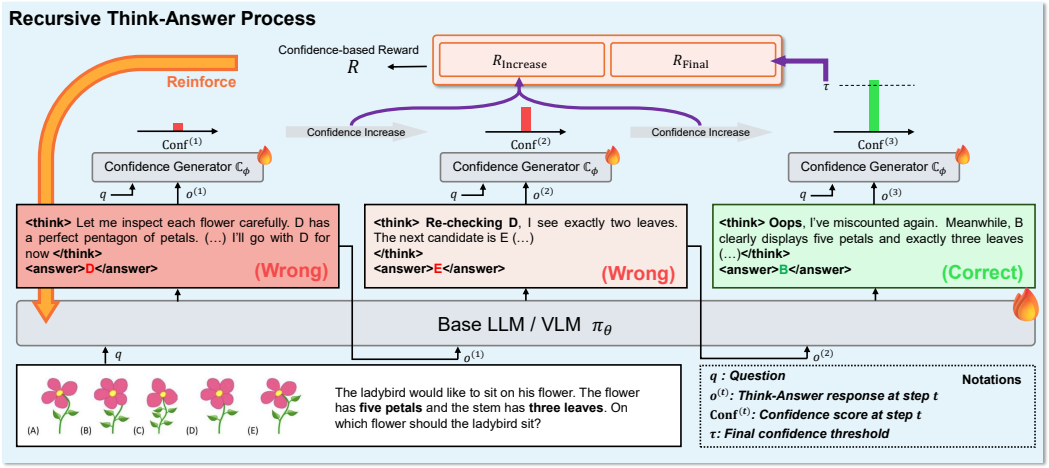
Answer process [16, 24, 40, 62, 66, 89, 102] to improve vision-language reasoning.

Despite these successes, current Think–Answer models almost always rely on a single-pass reasoning trajectory. After generating one Think–Answer pair, the model stops its inference process—even when the reasoning is inaccurate, inconsistent, or clearly uncertain. Models often produce self-reflective cues such as "Oops!" or "Let me try again" which show their uncertainty. However, these signals are not used: the model outputs its final answer without any method for self-evaluation or additional refinement. As a result, incorrect but confident-sounding reasoning remains uncorrected, reducing the reliability and consistency of Think–Answer models.

This limitation mainly comes from current reinforcement learning (RL) frameworks. Recent GRPO-style [70] sampling methods optimize only a single Think–Answer trajectory with rewards such as accuracy or format correctness. However, these methods do not consider the model's confidence in its reasoning, so they cannot support introspective checks or recursive correction. Because of this, the model cannot tell when its answer has low confidence and needs further refinement, which becomes a serious issue for complex or high-stakes tasks.

To address these limitations, we introduce R-TAP, **R**ecursive **T**hink-**A**nswer **P**rocess that enables LLMs and VLMs to iteratively refine their reasoning through confidence-guided Think–Answer cycles. Instead of stopping after one reasoning pass, the model evaluates its own confidence after each cycle using a dedicated *Confidence Generator*. When the models internally think the confidence is low, the model runs an additional Think–Answer cycle, revisiting and improving its previous reasoning. This recursive approach encourages deeper thinking, self-reflection, and step-by-step refinement of the reasoning process.

R-TAP introduces two key technical components. First is a Confidence Generator, initialized from the pretrained model and fine-tuned to quantify correctness of individual reasoning paths. Second is a recursive reward structure, combining (i) *Recursively Confidence Increase Reward* that encourages confidence to improve from one cycle to the next, and (ii) *Final Answer Confidence Reward* that encourages high-confidence final predictions. Together,



(Wrong)

Figure 4. **Recursive Think-Answer Process.** Given a question q, base LLM/VLM π_θ recursively generates multiple Think-Answers $o^{(t)}$ until the answer is correct $t = M$. In this example, effective recursion depth $M = 3$. A pre-trained Confidence Generator $\mathbb{C}_\phi$ assess each question and Think-Answer pair $(q, o^{(t)})$ then generates confidence score $\text{Conf}^{(t)}$. This confidence score is used to formulate confidence-based reward – R_{Increase} and R_{Final} – which serves as a sufficient reinforcement signal to train the model to recursively generate higher confidence Think-Answers until intrinsic confidence is high enough. Note that full responses for this question is described in Appendix A.

these components provide the necessary training signals for LLMs and VLMs to learn recursive Think-Answer trajectories for strengthening their reasoning.

To implement this approach, we proceed in two stages. Stage 1 performs supervised learning on the Confidence Generator using binary correctness labels for each reasoning trajectory produced by the target model. Stage 2 applies RL with GRPO to optimize the model's reasoning behavior under recursive rewards, enabling the model to generate progressively more accurate and confident reasoning across cycles. During this stage, the Confidence Generator is trained simultaneously to predict reliable confidence scores for the updated model's responses in real time. Notably, the Confidence Generator is used only during training, so R-TAP introduces no additional inference-time cost. In summary, R-TAP explicitly optimizes both introspection and self-correction, allowing models to refine their own trajectories and move beyond the rigid, one-shot Think-Answer paradigm.

Our experiments show that R-TAP delivers strong and consistent performance improvements across diverse language and vision-language reasoning benchmarks. Moreover, we find that recursive refinement leads to a substantial reduction in "Oops!"-style self-corrections during inference, indicating that R-TAP achieves more reliable yet fast inference-time reasoning with fewer failures along the trajectory. These results demonstrate that confidence-guided recursion training is a powerful mechanism for enhancing both the accuracy and inference speed of modern reasoning models. Our main contributions are summarized as follows:

- **Recursive Think-Answer Process**: We propose R-TAP, a confidence-driven iterative reasoning framework that enables dynamic re-engagement of reasoning cycles and self-corrective refinement.
- **Unified Reasoning Across Modalities**: R-TAP generalizes effectively to both language-only and multimodal reasoning, providing a unified mechanism for recursive improvement.

2. Related Work

Evolution of LLMs and VLMs. The rapid scaling of LLMs and VLMs has enabled impressive improvements in reasoning, alignment, and multimodal understanding. Early efforts such as GPT-3 [13] demonstrated strong in-context learning capabilities, while subsequent alignment-oriented approaches including InstructGPT [64] and ChatGPT leveraged RL from human feedback (RLHF) to improve reliability and adherence to instructions. More advanced closed-source models such as GPT-4 [6] expanded these capabilities to multimodal settings, showcasing strong performance across diverse problem-solving tasks.

Open-source research has paralleled these advances, driven by architectural innovations and large-scale instruction tuning. LLaMA [77] and its successors introduced lightweight yet performant architectures, while models such as LLaVA-NeXT [51], MM1 [60], Yi-VL [92], and MiniGemini [49] scaled VLM reasoning by increasing training data and integrating powerful vision encoders. Additional efforts incorporate specialized projectors or structured modules [15, 30, 76, 91] to extract hierarchical or task-specific features. Despite these advancements, most models still depend on single-pass predictions at inference time, without iterative introspection or self-correction.

Think-Answer Reasoning. Think–Answer paradigm has emerged as a compelling approach for enhancing reasoning in both LLMs and VLMs. Chain-of-Thought prompting [83] first demonstrated the benefits of eliciting inter-

Table 1. Performance comparison of open-source large language models on challenging math benchmarks: AIME24 [65], AMC [1], MATH500 [3], Minerva [4], and OlympiadBench [36].

Base Model + Method	AIME24	AMC	MATH500	Minerva	OlympiadBench	Avg.
Qwen2.5-Math-1.5B [88]	16.7	43.4	61.8	15.1	28.4	33.1
Qwen2.5-Math-1.5B-Instruct [88]	10.0	48.2	74.2	26.5	40.2	39.8
R1-Distill-Qwen-1.5B @ 3k [35]	2.5	21.7	52.2	16.3	17.3	22.0
R1-Distill-Qwen-1.5B @ 8k [35]	20.0	49.4	77.4	25.0	35.8	41.5
R1-Distill-Qwen-1.5B-R-TAP @ 8k	**27.0**	**55.8**	**83.5**	**31.9**	**42.4**	**48.1**
Oat-Zero-1.5B [55]	20.0	53.0	74.2	25.7	37.6	42.1
Oat-Zero-1.5B-R-TAP	**26.2**	**59.7**	**80.0**	**31.2**	**42.8**	**48.0**
OpenReasoner-Zero-7B @ 3k [37]	13.3	47.0	79.2	31.6	44.0	43.0
OpenReasoner-Zero-7B @ 8k [37]	13.3	54.2	82.4	31.6	47.9	45.9
Qwen2.5-Math-7B [88]	0.2	45.8	69.0	21.3	34.7	38.2
Qwen2.5-Math-7B-Instruct [88]	16.7	53.0	83.6	29.8	42.7	45.1
SimpleRL-Zero-7B [97]	26.7	60.2	78.2	27.6	40.3	46.6
PRIME-Zero-7B [23]	16.7	62.7	83.8	36.0	40.9	48.0
R1-Distill-Qwen-7B @ 3k [35]	10.0	26.2	60.1	23.0	23.1	28.5
R1-Distill-Qwen-7B @ 8k [35]	33.3	68.4	88.1	35.9	47.7	54.7
R1-Distill-Qwen-7B-R-TAP	**39.7**	**75.0**	**92.7**	**42.3**	**53.8**	**60.7**
Oat-Zero-7B [55]	43.3	62.7	80.0	30.1	41.0	51.4
Oat-Zero-7B-R-TAP	**50.5**	**69.5**	**87.2**	**37.2**	**46.8**	**57.7**

Table 2. Comparison of closed and open-source language models with MiMo-RL-7B-R-TAB across general knowledge, mathematical, and code benchmarks. General: GPQA Diamond [69], SuperGPQA [25], DROP [26], MMLU-Pro [82], IF-Eval [103]; Math: MATH500 [3], AIME2024 [65], AIME2025 [65]; Code: LiveCodeBench v5, v6 [43].

Benchmark	GPT-4o 0513 [41]	Claude-3.5-Sonnet-1022 [8]	OpenAI o1-mini [42]	QwQ-32B Preview [74]	R1-Distill-Qwen-14B [35]	R1-Distill-Qwen-7B [35]	MiMo-7B-RL [72]	**MiMo-RL 7B-R-TAP**
General								
GPQA Diamond (Pass@1)	49.9	**65.0**	60.0	54.5	59.1	49.1	54.4	60.7
SuperGPQA (Pass@1)	42.4	**48.2**	45.2	43.6	40.6	28.9	40.5	47.3
DROP (3-shot F1)	83.7	**88.3**	83.9	71.2	85.5	77.0	78.7	84.5
MMLU-Pro (EM)	72.6	78.0	**80.3**	52.0	68.8	53.5	58.6	65.9
IF-Eval (Prompt Strict)	84.3	**86.5**	84.8	40.4	78.3	60.5	61.0	68.0
Mathematics								
MATH500 (Pass@1)	74.6	78.3	90.0	90.6	93.9	92.8	95.8	**97.3**
AIME 2024 (Pass@1)	9.3	16.0	63.6	50.0	69.7	55.5	68.2	**75.8**
AIME 2025 (Pass@1)	11.6	7.4	50.7	32.4	48.2	38.8	55.4	**61.9**
Code								
LiveCodeBench v5 (Pass@1)	32.9	38.9	53.8	41.9	53.1	37.6	57.8	**64.2**
LiveCodeBench v6 (Pass@1)	30.9	37.2	46.8	39.1	31.9	23.9	49.3	**56.3**

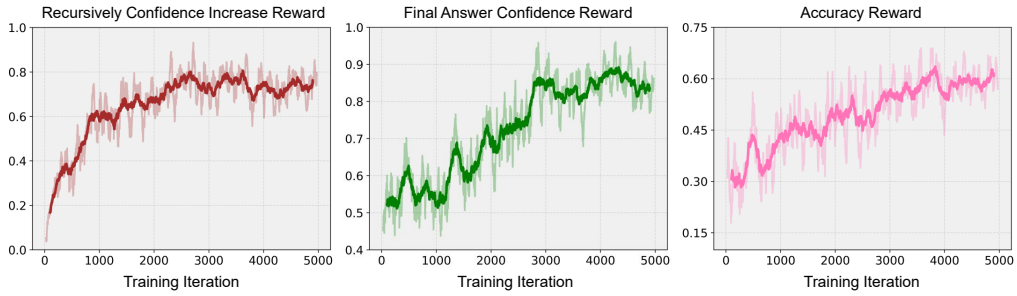


Figure 5. Training curves showing the progression of three reward signals—recursively confidence increase reward, last answer's confidence reward, and accuracy reward—over iterations during GRPO [70]. All rewards show consistent upward trends, indicating effective recursive refinement.

mediate reasoning steps, followed by extensions such as Program-of-Thoughts [18], Tree of Thoughts [90], and Graph of Thoughts [12] that explore larger reasoning spaces through structured search. While these methods improve robustness, they typically rely on external sampling or reranking rather than intrinsic self-evaluation.

Several reinforcement-learning-based methods introduce iterative refinement signals, including Reflexion [71] and Self-Consistency [80], but they operate outside the Think–Answer framework or depend on majority voting instead of introspective certainty. Recent Think-Answer models—DeepSeek-R1 [35] and OpenAI o1 [42]—demonstrate that separating "thinking" from "answering" can yield strong results in mathematical reasoning and programming. Similar trends are observed in multimodal reasoning, where models such as LMM-R1 [66], Vision-R1 [40], R1-V [16], and R1-Zero [102] adopt slow-thinking trajectories for visual problem solving.

However, almost all existing Think–Answer systems adopt a *single-pass* reasoning trajectory: after producing one Think–Answer pair, the model terminates inference even when its reasoning is uncertain or contains explicit self-reflective cues (e.g., "Oops!" and "Let me try again"). Sampling-based re-ranking strategies partially mitigate this issue but do not enable the model to internally assess confidence or decide whether further reasoning is needed.

Recursive and Confidence-Guided Refinement. Although iterative refinement has been explored through external verification [80] or heuristic feedback [71], prior works lack a principled mechanism to internally estimate the correctness of reasoning and dynamically invoke additional reasoning cycles. No existing Think–Answer model explicitly incorporates a confidence predictor to guide recursive inference-time reasoning.

In contrast, R-TAP introduces a confidence-aware recursive Think–Answer process that enables models to introspect, detect low-confidence reasoning, and selectively re-engage in additional reasoning cycles. By measuring both the recursive increase in confidence across reasoning cycles and the confidence of the final prediction, R-TAP offers a unified and efficient mechanism for self-corrective reasoning in both LLMs and VLMs, moving beyond the static, single-pass nature of prior approaches.

3. R-TAP: Recursive Think–Answer Process

R-TAP enables a model to perform confidence-guided recursive reasoning, allowing it to (i) evaluate the reliability of its reasoning, (ii) continue reasoning when uncertain, and (iii) terminate early when sufficiently confident. This section presents the overall framework, the Confidence Generator, the recursive generation scheme, the confidence-based reward design, and implementation details.

3.1. Problem Definition and Overall Framework

Given a question q, conventional single-step reasoning LLMs/VLMs produce exactly single Think-Answer pro-

Table 3. Performance comparison across various models including our proposed AZR variants on coding and mathematical reasoning benchmarks: HumanEval [17], MBPP [9], LiveCodeBench (LCB) [9], AIME [65], AMC [1], MATH500 [3], Minerva [4], and OlympiadBench [36].

Model	Base	HEval	MBPP	LCB	AME24	AME25	AMC	M500	Minerva	Olympiad	CAvg	MAvg	AVG
Base Models													
Qwen2.5-7B [87]	-	73.2	65.3	17.5	6.7	3.3	37.5	64.8	25.0	27.7	52.0	27.5	39.8
Qwen2.5-7B-Ins [87]	-	75.0	68.5	25.5	13.3	6.7	52.5	76.4	35.7	37.6	56.3	37.0	46.7
Qwen2.5-7B-Coder [87]	Coder	80.5	69.3	19.9	13.3	6.7	40.0	54.0	17.1	21.9	56.6	33.9	40.2
Qwen2.5-7B-Math [87]	-	61.0	57.9	16.2	10.0	16.7	42.5	64.2	15.4	28.0	45.0	29.5	37.3
Zero-Style Reasoners for Code													
AceCoder-RM [96]	Ins	79.9	71.4	23.6	20.0	6.7	50.0	76.4	34.6	36.7	58.3	37.4	47.9
AceCoder-Rule [96]	Ins	77.4	69.0	19.9	13.3	6.7	50.0	76.0	37.5	37.8	55.4	36.9	46.2
AceCoder-RM [96]	Coder	78.0	66.4	27.5	13.3	6.7	50.0	62.6	29.4	29.0	57.3	27.5	42.4
AceCoder-Rule [96]	Coder	80.5	70.4	29.0	6.7	6.7	37.5	62.8	27.6	27.4	60.0	28.5	44.8
CodeR1-LC2k [52]	Ins	81.7	71.7	28.1	13.3	3.3	40.0	74.0	35.6	36.7	60.3	35.8	48.6
CodeR1-12k [52]	Ins	81.1	73.5	29.9	13.3	3.3	37.5	74.0	35.7	36.9	61.3	33.5	47.4
PRIME-Zero [23]	Coder	49.4	51.1	11.0	23.3	**23.3**	**67.5**	**81.2**	37.9	41.8	37.2	45.8	41.5
AZR [101]	Coder	83.5	69.6	31.7	20.0	10.0	57.5	72.6	36.4	38.2	61.6	39.1	50.4
AZR-R-TAP	Coder	**89.3**	**75.9**	**38.2**	**27.4**	18.2	64.9	78.1	**44.3**	**46.6**	**67.8**	**46.6**	**57.2**
Zero-Style Reasoners for Math													
SimpleRL-Zoo [97]	Base	62.3	63.2	25.5	**20.0**	3.3	**67.5**	80.7	37.1	41.0	54.0	38.5	46.5
Oat-Zero [55]	Math	62.2	59.0	15.2	16.7	3.3	66.7	65.5	44.3	44.9	51.6	35.4	43.4
ORZ [38]	Base	80.5	64.3	22.0	13.3	16.7	60.0	81.8	32.7	**45.0**	55.6	**41.6**	48.6
AZR [101]	Base	71.3	69.1	25.3	13.3	13.3	52.5	74.4	38.2	38.2	55.2	39.0	46.8
AZR-R-TAP	Base	**78.5**	**76.7**	**32.4**	19.0	**20.1**	59.8	**81.2**	**45.7**	44.4	**62.5**	38.4	**50.5**

Table 4. Comparison of mathematical and coding benchmark performance across various instruction-tuned reinforcement-tuned and R-TAP integrated models. Benchmarks include AIME [65], AMC [1], MATH500 [3], MinervaMath [4], OlympiadBench [36], LeetCodeBench [22], and LiveCodeBench [43].

Method	AIME 2024	AMC	MATH500	MinervaMath	OlympiadBench	LeetCodeBench	LiveCodeBench	Avg.
GPT-4o [41]	9.3	45.8	76.4	36.8	43.3	**58.9**	**48.8**	45.6
Llama-3.1-70B-Inst. [32]	20.0	37.3	65.0	37.1	30.5	35.0	34.4	37.0
Qwen2.5-Math-7B-Inst. [87]	13.3	50.6	79.8	34.6	40.7	11.7	11.3	34.6
Eurus-2-7B-SFT [93]	3.3	30.1	66.2	32.7	29.8	21.7	17.8	28.8
RLOO [7]	20.0	47.0	73.2	36.4	35.4	28.3	26.7	36.9
Eurus-2-7B-PRIME [93]	20.0	50.6	78.2	39.3	40.3	31.1	27.5	41.0
Eurus-2-7B-PRIME-R-TAP	**28.3**	**57.5**	**83.5**	**43.8**	**47.4**	38.6	31.8	**47.2**

Table 5. GRPO-trained models on MathVerse [100], MathVision [78], MathVista [58], WeMath [67], and HallusionBench [50].

Model	MathVerse	MathVision	MathVista	WeMath	HallusionBench	Avg.
R1-VL-7B [98]	40.0	24.7	63.5	-	-	-
Vision-R1-7B [40]	52.4	-	73.5	-	-	-
R1-OneVision-7B [89]	46.1	22.5	63.9	62.1	65.6	52.0
OpenVLThinker-7B [24]	48.0	25.0	71.5	67.8	70.8	56.5
MM-Eureka-Qwen-7B [61]	50.5	28.3	71.5	65.5	68.3	56.8
ADORA-7B [33]	50.1	27.6	71.1	67.1	53.1	53.8
ThinkLite-7B-VL [81]	50.2	27.6	72.7	69.2	71.0	58.1
VLAA-Thinker-Qwen2.5-7B [14]	49.9	26.9	68.8	67.9	68.6	56.4
Qwen2.5-VL-7B-Instruct [10]	46.2	25.0	67.5	63.1	71.2	53.3
+ Vanilla GRPO [70]	50.7	28.5	71.7	68.6	69.8	57.9
+ NoisyRollout [54]	52.8	28.9	72.9	71.9	70.8	59.5
+ **R-TAP**	**60.1**	**35.3**	**79.4**	**78.2**	**77.9**	**66.2**

cess:

$$o^{(1)} \sim \pi_\theta(o \mid q), \tag{1}$$

and terminate immediately, even when the produced reasoning is uncertain or incorrect. Such models lack an internal mechanism for introspection or self-initiated refinement.

R-TAP generalizes this paradigm by allowing the model to recursively generate recursive Think-Answer responses $\mathcal{O}$ as follows:

$$\mathcal{O} = \{o^{(1)}, o^{(2)}, \ldots, o^{(T)}\} \sim \pi_\theta(\mathcal{O} \mid q), \quad \text{such that} \quad o^{(t+1)} \sim \pi_\theta(o \mid q, \{o^{(i)}\}_{i=1}^{t}), \tag{2}$$

where T denotes the recursion depths. During training, T is fixed to do efficient batch sampling, while at inference time the model internally determines whether to continue or terminate.

Similar to prior single-pass Think-Answer models [35], R-TAP also trains the model by maximizing the objective of GRPO [70] based on G generated samples, which can be written as follows:

$$\max_\theta \mathbb{E}_{q\sim\mathcal{D},\{\mathcal{O}_i\}_{i=1}^{G}\sim\pi_\theta(\mathcal{O}|q)}[\mathbb{E}_i[\min\left(r_i A_i, \text{clip}(r_i, 1-\epsilon, 1+\epsilon)A_i\right) - \beta\mathcal{D}_{\text{KL}}(\pi_\theta\|\pi_{\text{ref}})]], \tag{3}$$

where the policy ratio r_i and advantage A_i are defined by

$$r_i = \frac{\pi_\theta(\text{O}_i \mid q)}{\pi_{\theta_{\text{old}}}(\text{O}_i \mid q)}, \qquad A_i = \frac{R_i - \mathbb{E}[\{R_i\}_{i=1}^{G}]}{\sigma[\{R_i\}_{i=1}^{G}]}, \tag{4}$$

with R_i denotes the total reward for recursive trajectory $\mathcal{O}_i$ which will be explained in next section. This objective encourages the model to engage in further recursive reasoning when it internally detects low confidence, while allowing it to stop once its confidence reaches a satisfactory level.

3.2. Confidence Generator

A central component of R-TAP is the *Confidence Generator* $\mathbb{C}_\phi$, which provides the reliability of each response in recursive Think-Answer trajectory. Importantly, $\mathbb{C}_\phi$ is used only during R-TAP training and removed at inference, so R-TAP introduces no inference-time cost.

Architecture. Given a question q and one of Think-Answer responses: $o^{(t)}$ in Eq. (2), the Confidence Generator $\mathbb{C}_\phi$ outputs a scalar confidence score from zero to one continuous value:

$$\text{Conf}^{(t)} = \mathbb{C}_\phi(q, o^{(t)}). \tag{5}$$

We build Confidence Generator from π_{ref} structure in the beginning but replace the language head with a confidence head $\mathbb{R}^{d\times 1}$ (d: hidden dimension), followed by a sigmoid activation.

Supervised pre-training. Prior to R-TAP training, we train $\mathbb{C}_\phi$ to predict confidence of the given response. To implement it, we utilize the binary classification for whether the response is correct compared with ground-truth label. For each question q, the model generates N single Think-Answer process samples. Let $\{o_i^{\text{correct}}\}_{i=1}^{K}$ denote samples with correct answers and $\{o_i^{\text{wrong}}\}_{i=1}^{N-K}$ denote incorrect ones. We optimize $\mathbb{C}_\phi$ using binary classification objective:

$$\max_\phi[\frac{1}{K}\sum_{i=1}^{K}\log\mathbb{C}_\phi(q, o_i^{\text{correct}}) + \frac{1}{N-K}\sum_{i=1}^{N-K}\log\big(1-\mathbb{C}_\phi(q, o_i^{\text{wrong}})\big)], \tag{6}$$

where we technically set $K \approx N/2$ for balanced training. This step equips $\mathbb{C}_\phi$ with the ability to evaluate the reliability of recursive Think-Answer trajectories.

3.3. Confidence-Based Reward Design

R-TAP introduces two complementary confidence-driven rewards that (1) encourage refinement when necessary and (2) encourage termination when sufficiently confident.

Table 6. Performance of closed-source and open-source multimodal models on the R1-OneVision benchmark [89]. The benchmark is organized by diverse education level and subject category.

Model	Avg	Grade			Category					
		Junior High School	High School	College	Social Test	Math	Physics	Chemistry	Biology	Deduction
Closed-source										
GPT-4o [41]	49.6	51.3	56.2	45.3	26.5	41.3	52.5	71.4	63.4	26.5
Gemini-2.0-Flash [73]	59.1	56.0	65.9	61.2	39.8	52.3	64.4	74.3	67.2	39.8
Claude-3.5 [8]	52.1	56.0	55.9	49.4	30.6	46.5	54.3	66.7	65.7	30.6
Open-source										
MiniCPM-o-2.6 [39]	30.4	33.4	31.7	21.2	31.6	24.2	31.7	30.5	41.8	31.6
InternVL2.5-8B [19]	29.5	33.1	30.6	21.8	27.6	26.3	24.8	32.4	46.3	27.6
InternVL2.5-8B-MPO [19]	32.5	37.4	33.6	24.7	26.5	28.7	29.9	41.0	44.8	26.5
Qwen2-VL-7B [79]	30.0	35.4	32.5	25.9	26.5	26.3	28.1	30.5	45.5	26.5
Qwen2.5-VL-7B [79]	32.1	35.3	37.1	25.9	19.4	31.5	27.3	39.0	47.0	19.4
DeepSeek-VL2 [85]	29.8	34.4	30.9	18.8	30.6	23.5	28.4	29.5	47.8	30.6
R1-Onevision-7B [89]	36.2	40.1	39.5	27.6	26.5	33.0	30.2	49.5	53.0	26.5
R1-Onevision-7B-R-TAP	**42.4**	**47.4**	**45.0**	**34.9**	**31.4**	**39.4**	**36.9**	**55.9**	**59.6**	**31.2**
Qwen2.5-VL-72B [10]	52.0	54.3	56.7	54.1	23.5	48.9	55.8	63.8	63.4	23.5

Table 7. Comparing the performances on AIME2024 [65], AIME2025 [65], LiveCodeBench [43], and Arena-Head [75].

Model	AIME2024	AIME2025	LiveCodeBench (v5, 2024.10–2025.02)	Arena-Hard
AM-Thinking-v1 (Dense, 32B) [44]	85.3	74.4	70.3	92.5
AM-Thinking-v1-R-TAP (Dense, 32B)	**90.1**	**79.6**	**76.5**	**94.3**
Qwen3-235B-A22B (MoE, 235B) [86]	85.7	81.5	70.7	95.6
Qwen3-235B-A22B-R-TAP (MoE, 235B)	**90.9**	**88.7**	**77.6**	**97.1**
Qwen3-32B (Dense, 32B) [86]	81.4	72.9	65.7	93.8
Qwen3-32B-R-TAP (Dense, 32B)	**87.9**	**78.2**	**71.0**	**95.6**
DeepSeek-R1 (MoE, 671B) [35]	79.8	70.0	64.3	93.2
Nemetron-Ultra-253B (Dense, 256B) [11]	80.8	72.5	68.1	87.0
OpenAI-o1 (2024-12-17) [42]	74.3	79.2	63.9	92.1
OpenAI-o3-mini (Medium) [63]	79.6	74.8	66.3	89.0
Gemini2.5-Pro [73]	92.0	86.7	70.4	96.4

Table 8. Comparison of closed-source, general-purpose open-source, reasoning-focused, and proposed models on math-related benchmarks: MathVista [58], MathVerse [100], MathVision [78], OlympiadBench [36], WeMath [67].

Model	MathVista	MathVerse	MathVision	OlympiadBench	WeMath
Closed-Source Models					
Claude3.7-Sonnet [8]	66.8	52.0	41.3	48.9	72.6
GPT-4o [41]	63.8	50.2	30.4	35.0	68.8
o1 [42]	73.9	57.0	60.3	68.0	98.7
Gemini2-flash [73]	70.4	59.3	41.3	51.0	71.4
Open-Source General Models					
InternVL2.5-VL-8B [19]	64.4	39.5	19.7	12.3	53.5
Qwen-2.5-VL-7B [10]	68.2	47.9	25.4	20.2	62.1
InternVL2.5-VL-38B [19]	71.9	49.4	31.8	32.0	67.5
Qwen-2.5-VL-32B [10]	74.7/71.7	49.9	40.1	30.0	69.1
InternVL2.5-VL-78B [19]	72.3	51.7	32.2	31.1	66.3
Qwen-2.5-VL-72B [10]	74.8	57.6	38.1	40.4	72.4
Open-Source Reasoning Models					
InternVL2.5-8B-MPO [19]	68.9	35.5	21.5	7.8	53.5
InternVL2.5-38B-MPO [19]	73.8	46.5	32.3	25.6	66.2
QVQ-72B-Preview [73]	71.4	48.2	35.9	33.2	65.4
ADORA-7B [33]	73.5	50.1	23.0	20.1	64.2
R1-Onevision-7B [89]	64.1	47.1	29.9	17.3	61.8
OpenVLThinker-7B [24]	70.2	47.9	25.3	20.1	64.3
MM-Eureka-7B [61]	73.0	50.3	26.9	20.1	66.1
MM-Eureka-7B-R-TAP	**79.3**	**56.1**	**31.7**	**27.5**	**71.0**
MM-Eureka-32B [61]	74.8	56.5	34.4	35.9	73.4
MM-Eureka-32B-R-TAP	**80.2**	**61.8**	**39.9**	**41.2**	**79.3**

Recursive confidence increase reward. To reward meaningful refinement across recursive steps, we define:

$$R_{\text{Increase}} = \frac{1}{M-1} \sum_{t=1}^{M-1} \mathbb{1}\left[\text{Conf}^{(t+1)} > \text{Conf}^{(t)}\right], \quad (7)$$

where M is the effective recursion depth. For example, if the third Think-Answer response is correct but the previous ones are not, then $M = 3$ is satisfied, but if the first Think-Answer response is correct, because $M = 1$, we define this case to one reward value. Note that M cannot be larger than T such that $M \leq T$ because we pre-define T for effective batch sampling to generate recursive Think-Answer responses.

Final answer confidence reward. The final answer must be sufficiently confident:

$$R_{\text{Final}} = \mathbb{1}\left[\text{Conf}^{(M)} \geq \tau\right], \quad (8)$$

where τ is a preset threshold.

Combined reward. The total reward is simply calculated by the addition, which is written as follows

$$R = R_{\text{Increase}} + R_{\text{Final}} + R_{\text{Format}} + R_{\text{Answer}} + R_{\text{Length}}, \quad (9)$$

where we equally use the conventional rewards used in Open-R1 [28]. R_{Format} is the reward for Think-Answer format, R_{Answer} is for model response' correctness, and R_{Length} is for a soft length penalty. Appendix B describes the algorithms for R-TAP.

In summary, R-TAP provides broadly applicable mechanism for any LLMs and VLMs to enable recursive, confidence-guided reasoning within existing Think–Answer architectures. Furthermore, since the Confidence Generator is used only during training, R-TAP preserves the inference-time efficiency of standard single-pass models while equipping them with the ability to internally assess uncertainty and selectively refine their own reasoning for more stable yet faster. This makes R-TAP a scalable and practical solution for improving the stability, reliability, and depth of reasoning in modern models—bridging the gap between rigid single-pass inference and fully self-corrective reasoning systems.

4. Experiments

4.1. Implementation Details

We conduct training and evaluation of R-TAP primarily on NVIDIA A100 80GB GPUs. To ensure fast text generation during training, we employ vLLM [47], which is built on PagedAttention. In the pre-training step of Confidence Generator, vLLM [47] is used to generate $N = 128$ responses for each question for both LLMs and VLMs. We train the Confidence Generator under DeepSpeed engine with ZeRO-3 [68] with AdamW optimizer [57], applying a linearly decayed learning rate from 1e-5 to 1e-6 to pre-train the confidence generator. Next step is training LLMs/VLMs as well as the Confidence Generator, where we also leverage vLLM [47] for online text generation and use DeepSpeed with ZeRO-3 to update both models with a fixed learning rate of 1e-6. During this phase, we use $T = 4$ recursion depths, for which we generate $G = 12$ response outputs. From this setting, we update target models for 12 GRPO [70] iterations per each training iteration by using $\epsilon = 0.2$ and $\beta = 0.04$ in Eq. (3). For diverse responses, we set the generation hyperparameters to temperature=1.0, top-p=0.95, top-k=50, and repetition penalty=1.05. In addition, we set τ to 0.55 and we find its optimal τ by doing greedy

Table 9. Performance of Llama3.1 [32], DeepSeek-Math [70], Mistral [45], and Qwen-2.5 [87] models before and after SimpleRL-Zoo [97] fine-tuning and R-TAP integration across various mathematical reasoning benchmarks. Benchmarks include GSM8K [20], MATH500 [3], Minerva [4], OlympiadBench [36], AIME [65], and AMC [1]. Note that, the selection of evaluation benchmarks and their performance results, and the selection of the compared models are referred to the manuscript of SimpleRL-Zoo [97].

Model	GSM8K	MATH500	Minerva Math	Olympiad Bench	AIME24 (P@1)	AIME24 (Avg@32)	AMC23
Llama, DeepSeek and Mistral Models							
Mistral-v0.1-7B [45]	21.2	4.2	4.0	2.4	0.0	0.0	0.0
+ SimpleRL-Zoo	75.0	15.8	6.6	4.1	0.0	0.2	10.0
+ R-TAP	**81.2**	**21.3**	**13.9**	**10.0**	**6.4**	**9.5**	**17.4**
Llama-3.1-8B [27]	39.7	13.6	4.8	3.1	0.0	0.2	2.5
+ SimpleRL-Zoo	79.2	23.0	9.6	5.3	0.0	0.2	15.0
+ R-TAP	**86.5**	**30.4**	**16.3**	**13.7**	**6.7**	**7.9**	**22.3**
DeepSeek-Math-7B [70]	28.4	19.4	5.5	4.7	0.0	0.0	10.0
+ SimpleRL-Zoo	78.5	39.6	21.0	12.6	3.3	0.6	20.0
+ R-TAP	**83.7**	**45.2**	**28.4**	**20.1**	**10.2**	**8.2**	**27.5**
Mistral-Small-24B [45]	78.6	43.6	10.7	11.6	3.3	0.5	17.5
+ SimpleRL-Zoo	92.0	70.6	36.8	36.6	16.7	13.1	45.0
+ R-TAP	**95.4**	**77.7**	**40.2**	**43.2**	**23.4**	**18.5**	**51.1**
Qwen Series Models							
Qwen-2.5-0.5B [87]	36.7	15.8	4.8	2.8	0.0	0.3	12.5
+ SimpleRL-Zoo	49.5	34.4	10.3	8.9	0.0	0.7	22.5
+ R-TAP	**54.3**	**39.8**	**16.6**	**14.2**	**6.4**	**6.9**	**28.3**
Qwen-2.5-1.5B [87]	55.7	29.6	6.6	6.5	0.0	0.1	12.5
+ SimpleRL-Zoo	74.4	59.0	20.2	21.0	6.7	4.2	35.0
+ R-TAP	**79.2**	**64.4**	**27.1**	**28.2**	**13.0**	**11.5**	**41.7**
Qwen-2.5-7B [87]	88.2	64.6	25.7	30.1	3.3	0.3	30.0
+ SimpleRL-Zoo	91.7	78.2	38.6	40.4	20.0	15.6	62.5
+ R-TAP	**96.0**	**83.5**	**44.8**	**46.9**	**28.1**	**22.3**	**69.4**
Qwen-2.5-Math-7B [88]	86.5	63.6	12.5	25.8	13.3	8.6	42.5
+ SimpleRL-Zoo	90.2	80.2	37.5	39.0	40.0	24.0	70.0
+ R-TAP	**94.2**	**86.7**	**42.3**	**46.1**	**45.6**	**29.3**	**78.1**
Qwen-2.5-14B [87]	91.6	65.4	24.3	33.5	6.7	3.4	37.5
+ SimpleRL-Zoo	94.4	80.2	40.4	44.9	23.3	14.2	57.6
+ R-TAP	**97.8**	**86.3**	**48.1**	**52.3**	**30.5**	**19.8**	**61.7**
Qwen-2.5-32B [87]	92.9	68.6	27.9	31.1	10.0	4.5	45.0
+ SimpleRL-Zoo	95.9	82.4	42.6	46.4	36.7	27.2	67.5
+ R-TAP	**97.9**	**87.9**	**47.3**	**52.3**	**42.9**	**32.0**	**71.8**

search from zero to one value by 0.05 points. For evaluation, we remove the confidence generator and use only the trained LLMs/VLMs. We keep the default generation hyperparameters of backbone LLMs/VLMs.

4.2. Validating R-TAP

For LLMs, we train all the parameters, but for VLMs, we do not train vision encoder parts due to some observations of performance degradation. For selecting training dataset, we employ same training dataset on which LLMs/VLMs is trained for think-answer process, except some LLMs that do not release their own training dataset such as Phi-4-reasoning series [5], MiMo [72], Qwen3 [86], and AM-Thinking-v1 [44]. For these models, we instead gather Open-R1-Math (220K) and codeforce-cot (100K) in Open-R1 [28], and PRIME (481K) [23] covering math and code as well, and train them by using algorithm of R-TAP.

As shown in Fig. 1, R-TAP-applied Phi-4-reasoning models have shown dramatic improvements, thereby closing to OpenAI-o1 and -o3 models. Figure 5 represents the reward graph of Phi-4-reasoning-plus [5] during training R-TAP, indicating stable training. To see more generalizability and applicability, we employ several LLMs and their dataset. First, we consider R1-Distill-Qwen-1.5B and -7B [35] and Oat-Zero-1.5B, -7B [56], AZR-Base-7B and AZR-Coder-7B [101] as backbone models and we train them by R-TAP on Open-Reasoner-Zero-57K [37]. In addition, we apply R-TAP to SimpleRL-Zoo [97] in Tab. 9 and PRIME [23] by using each their own curated data (57K and 481K, respectively). Remarkably, Tab. 1-Tab. 4 shows dramatic improvements of LLMs with R-TAP as well. We additionally employ recently released LLMs: Qwen3 [86] and AM-Thinking-v1 [44], and we observe their consistent dramatic performance improvements by R-TAP in Tab. 7, compared with state-of-the-art open- and closed-source reasoning LLMs.

To extend its effectiveness to VLMs, we apply R-TAP to Skywork-R1V2 [84] by Skywork-OR1-RL-120K, Geometry-3K, R1-OneVision-155K, and MMK12-16K. Notably, Fig. 2 shows its consistent performance improvements despite multimodality. Besides, we employ Noisy-Rollout-7B [54], R1-OneVision-7B [89], and MM-Eureka-7B and -32B [61] and train them on their own training dataset: Geometry-3K, R1-OneVision-155K, and MMK12-16K. Tab. 5, Tab. 6, and Tab. 8 shows their performance improvements as well.

4.3. Ablation Studies

We conduct comprehensive ablation studies to validate the effect of each R-TAP component in Tab. 10 and Tab. 11, superiority of confidence estimation method in Tab. 12 and Tab. 13, and the effect of recursion depths in Tab. 14 and Tab. 15. In this study, we deal with Phi-4-reasoning-plus [5] for LLM and Skywork-R1V2 [84] for VLM. We analyze the effect of removing confidence increase reward R_{Increase}, final answer confidence reward R_{Final} and answer reward R_{Answer}. Tab. 10 and Tab. 11 show that all components of R-TAP are essential for achieving highest accuracy in both language-only and multi-modal settings.

We further show the superiority of our Confidence Generator design by comparing with four recent confidence estimation approaches: (a) Calibration via ECE [31, 34] (b) Se-

Table 10. Effect of each R-TAP component on LLM

Method	$\mathbb{C}_\phi$	R_{Increase}	R_{Final}	R_{Answer}	AIME25	HMMT Feb25	OmniMath	GPQA	LiveCodeBench	Avg
Baseline	✗	✗	✗	✓	78.0	53.6	81.9	69.3	65.9	69.7
R-TAP	✓	✓	✗	✓	80.1	56.2	84.0	71.0	70.1	72.3
R-TAP	✓	✗	✓	✓	81.5	57.8	85.1	73.2	72.0	73.9
R-TAP	✓	✓	✓	✓	**83.7**	**60.3**	**86.2**	**76.7**	**72.1**	**75.8**
R-TAP	✓	✓	✓	✗	61.2	44.7	59.9	53.0	51.8	56.1

Table 11. Effect of each R-TAP component on VLM

Method	$\mathbb{C}_\phi$	R_{Increase}	R_{Final}	R_{Answer}	MMMU	MathVista	OlympiadBench	MathVision	MMMU-Pro	Avg
Baseline	✗	✗	✗	✓	73.6	74.0	62.6	49.0	52.0	62.2
R-TAP	✓	✓	✗	✓	76.1	78.0	66.4	53.0	54.4	65.4
R-TAP	✓	✗	✓	✓	77.8	80.2	68.3	55.3	58.1	67.9
R-TAP	✓	✓	✓	✓	**78.2**	**82.3**	**69.4**	**56.8**	**59.2**	**69.2**
R-TAP	✓	✓	✓	✗	53.2	50.7	45.9	41.3	52.4	48.7

Table 12. Confidence Estimation Methods for LLM

LLMs	AIME25	HMMT Feb25	OmniMath	GPQA	LCB	Avg
Phi-4-reasoning-plus	78.0	53.6	81.9	69.3	65.9	69.7
Phi-4-reasoning-plus (a)	81.5	57.0	85.7	73.0	69.3	73.3
Phi-4-reasoning-plus (b)	83.0	55.8	86.5	73.5	68.7	73.5
Phi-4-reasoning-plus (c)	82.5	57.0	87.2	72.8	70.5	74.0
Phi-4-reasoning-plus (d)	**84.2**	57.3	**87.5**	75.1	70.4	74.9
Phi-4-reasoning-plus-R-TAP	83.7	**60.3**	86.2	**76.7**	**72.1**	**75.8**

Table 13. Confidence estimation methods for VLM

VLMs	MMMU	MathVista	OlympiadBench	MathVision	MMMU-Pro	Avg
R1V2-38B	73.6	74.0	62.6	49.0	52.0	62.2
R1V2-38B (a)	75.1	78.0	65.8	53.5	57.6	66.0
R1V2-38B (b)	76.3	77.5	67.2	53.0	58.5	66.5
R1V2-38B (c)	77.0	79.0	68.5	54.2	57.3	67.2
R1V2-38B (d)	**79.5**	80.3	**70.8**	**58.2**	55.7	68.9
R1V2-38B-R-TAP	78.2	**82.3**	69.4	56.8	**59.2**	**69.2**

Table 14. Recursion depths for LLM

LLMs	T	AIME25	HMMT Feb25	OmniMath	GPQA	LiveCodeBench	Avg
Phi-4-reasoning-plus	1	78.0	53.6	81.9	69.3	65.9	69.7
Phi-4-reasoning-plus	2	78.7	54.2	82.4	70.4	67.3	71.0
Phi-4-reasoning-plus	3	79.2	55.0	82.8	71.0	67.9	71.2
Phi-4-reasoning-plus	4	80.3	56.4	83.6	72.5	69.1	72.5
Phi-4-reasoning-plus-R-TAP	2	80.9	57.1	84.1	73.3	68.6	72.6
Phi-4-reasoning-plus-R-TAP	3	82.3	58.9	85.0	75.1	70.3	74.9
Phi-4-reasoning-plus-R-TAP	4	**83.7**	**60.3**	**86.2**	**76.7**	**72.1**	**75.8**

Table 15. Recursion depths for VLM

VLMs	T	MMMU	MathVista	OlympiadBench	MathVision	MMMU-Pro	Avg
R1V2-38B	1	73.6	74.0	62.6	49.0	52.0	62.2
R1V2-38B	2	74.3	75.1	63.3	50.4	53.1	63.0
R1V2-38B	3	74.8	75.7	64.0	51.1	53.9	63.4
R1V2-38B	4	75.6	76.6	64.9	52.2	54.6	64.0
R1V2-38B-R-TAP	2	76.1	77.0	65.3	52.7	55.0	64.1
R1V2-38B-R-TAP	3	77.5	79.4	67.2	54.9	57.8	67.4
R1V2-38B-R-TAP	4	**78.2**	**82.3**	**69.4**	**56.8**	**59.2**	**69.2**

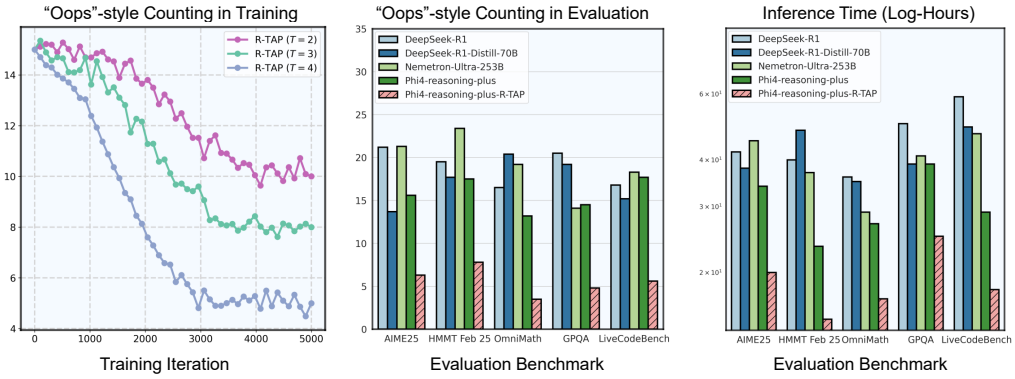


Figure 6. Impact of R-TAP on reducing the number of "Oops"-style words – which corresponds to the number of erroneous reasoning – and its effect on substantially reducing inference time. (Left) Negative correlation between the number of erroneous reasoning and R-TAP train iterations. (Center) Evaluation result on the significant reduction of erroneous reasoning by applying R-TAP. (Right) Evaluation result on the substantial reduction of inference time due to the reduction of erroneous reasoning by applying R-TAP.

mantic Equivalence Entropy [46] (c) "IDK" token [21] and (d) Iterative LLM generation [48, 99]. Tab. 12 and Tab. 13 show that applying our method achieves highest accuracy. Tab. 14 and Tab. 15 suggest that increasing recursion depths of Think-Answer from 1 to 4 times makes higher accuracy.

Finally, we additionally analyze how R-TAP affects the reduction of erroneous reasoning behaviors and inference efficiency, as summarized in Fig. 6. The left plot shows that the number of "Oops"-style tokens—our proxy for erroneous reasoning—monotonically decreases as training progresses, and deeper Think–Answer recursion (larger T) leads to faster and greater reduction. The center plot demonstrates that this trend generalizes to evaluation benchmarks: R-TAP consistently lowers erroneous reasoning across all datasets compared to baseline and the other state-of-the-art models. In addition, the right plot reveals that this reduction in reasoning errors directly translates to substantial improvements in inference efficiency, yielding significantly shorter inference time (log-hours) across all benchmarks. These results confirm that R-TAP not only improves accuracy but also makes reasoning more stable and computationally efficient. Besides, we describe detailed comparison between performance and computation complexity about decoding token count and training time in Appendix C.

5. Discussion and Conclusion

We introduced **R-TAP**, a method that augments LLMs and VLMs with a learned confidence generator and two rewards: recursive confidence increase and final-answer confidence reward. These components enable models to iteratively refine their think–answer trajectories, yielding substantial performance gains and narrowing the gap between smaller models and much larger ones. R-TAP supports up to T recursive steps. However, due to the batch-dependent nature of current deep-learning implementations, all T trajectories must be generated in parallel even when a confident answer emerges early. This simplifies parallelization but introduces significant computational and memory overhead. We hope that R-TAP encourages further research on confidence-aware iterative reasoning and supports the development of more efficient, trustworthy, and general-purpose reasoning systems for both LLMs and VLMs.

References

[1] Amc-23. https://huggingface.co/datasets/knoveleng/AMC-23. Accessed: 2025-05-16. 4, 5, 7

[2] Hmmt. https://www.hmmt.org/. Accessed: 2025-05-16. 1

[3] Math-500. https://huggingface.co/datasets/HuggingFaceH4/MATH-500. Accessed: 2025-05-16. 4, 5, 7

[4] Minerva-math. https://huggingface.co/datasets/knoveleng/Minerva-Math. Accessed: 2025-05-16. 4, 5, 7

[5] Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, Gustavo de Rosa, Suriya Gunasekar, Mojan Javaheripi, Neel Joshi, et al. Phi-4-reasoning technical report. *arXiv preprint arXiv:2504.21318*, 2025. 7

[6] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 3

[7] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024. 5

[8] Anthropic. The claude 3 model family: Opus, sonnet, haiku. https://www.anthropic.com, 2024. 4, 6

[9] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. 5

[10] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 5, 6

[11] Akhiad Bercovich and Itay Levy et al. Llama-nemotron: Efficient reasoning models, 2025. 6

[12] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 17682–17690, 2024. 4

[13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 3

[14] Hardy Chen, Haoqin Tu, Fali Wang, Hui Liu, Xianfeng Tang, Xinya Du, Yuyin Zhou, and Cihang Xie. Sft or rl? an early investigation into training r1-like reasoning large vision-language models. *arXiv preprint arXiv:2504.11468*, 2025. 5

[15] Kaibing Chen, Dong Shen, Hanwen Zhong, Huasong Zhong, Kui Xia, Di Xu, Wei Yuan, Yifei Hu, Bin Wen, Tianke Zhang, et al. Evlm: An efficient vision-language model for visual understanding. *arXiv preprint arXiv:2407.14177*, 2024. 3

[16] Liang Chen, Lei Li, Haozhe Zhao, Yifan Song, and Vinci. R1-v: Reinforcing super generalization ability in vision-language models with less than $3. https://github.com/Deep-Agent/R1-V, 2025. Accessed: 2025-02-02. 2, 4

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021. 5

[18] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022. 4

[19] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024. 6

[20] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 7

[21] Roi Cohen, Konstantin Dobler, Eden Biran, and Gerard de Melo. I don't know: Explicit modeling of uncertainty with an [idk] token. *Advances in Neural Information Processing Systems*, 37:10935–10958, 2024. 8

[22] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, pages 79–89, 2024. 5

[23] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025. 4, 5, 7

[24] Yihe Deng, Hritik Bansal, Fan Yin, Nanyun Peng, Wei Wang, and Kai-Wei Chang. Openvlthinker: An early ex-

ploration to complex vision-language reasoning via iterative self-improvement. *arXiv preprint arXiv:2503.17352*, 2025. 2, 5, 6

[25] Xinrun Du, Yifan Yao, Kaijing Ma, Bingli Wang, Tianyu Zheng, King Zhu, Minghao Liu, Yiming Liang, Xiaolong Jin, Zhenlin Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025. 1, 4

[26] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019. 4

[27] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 7

[28] Hugging Face. Open r1: A fully open reproduction of deepseek-r1, 2025. 6, 7

[29] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, et al. Omni-math: A universal olympiad level mathematic benchmark for large language models. *arXiv preprint arXiv:2410.07985*, 2024. 1

[30] Chunjiang Ge, Sijie Cheng, Ziming Wang, Jiale Yuan, Yuan Gao, Jun Song, Shiji Song, Gao Huang, and Bo Zheng. Convllava: Hierarchical backbones as visual encoder for large multimodal models. *arXiv preprint arXiv:2405.15738*, 2024. 3

[31] Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. A survey of confidence estimation and calibration in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6577–6595, 2024. 7

[32] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 5, 7

[33] Lujun Gui and Qingnan Ren. Training reasoning model with dynamic advantage estimation on reinforcement learning, 2025. Notion Blog. 5, 6

[34] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International conference on machine learning*, pages 1321–1330. PMLR, 2017. 7

[35] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 1, 4, 5, 6, 7

[36] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024. 2, 4, 5, 6, 7

[37] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025. 4, 7

[38] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model, 2025. 5

[39] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024. 6

[40] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025. 2, 4, 5

[41] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 4, 5, 6

[42] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024. 1, 4, 6

[43] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. 1, 4, 5, 6

[44] Yunjie Ji, Xiaoyu Tian, Sitong Zhao, Haotian Wang, Shuaiting Chen, Yiping Peng, Han Zhao, and Xiangang Li. Am-thinking-v1: Advancing the frontier of reasoning at 32b scale, 2025. 6, 7

[45] Fengqing Jiang. Identifying and mitigating vulnerabilities in llm-integrated applications. Master's thesis, University of Washington, 2024. 7

[46] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664*, 2023. 8

[47] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. 6

[48] Moxin Li, Wenjie Wang, Fuli Feng, Fengbin Zhu, Qifan Wang, and Tat-Seng Chua. Think twice before trusting: Self-detection for large language models through comprehensive answer reflection. *arXiv preprint arXiv:2403.09972*, 2024. 8

[49] Yanwei Li, Yuechen Zhang, Chengyao Wang, Zhisheng Zhong, Yixin Chen, Ruihang Chu, Shaoteng Liu, and Jiaya Jia. Mini-gemini: Mining the potential of multi-modality vision language models. *arXiv preprint arXiv:2403.18814*, 2024. 3

[50] Fuxiao Liu, Tianrui Guan, Zongxia Li, Lichang Chen, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. Hallusionbench: You see what you think? or you think what you see? an image-context reasoning benchmark challenging for gpt-4v (ision), llava-1.5, and other multi-modality models. *arXiv preprint arXiv:2310.14566*, 2023. 5

[51] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. 3

[52] Jiawei Liu and Lingming Zhang. Code-r1: Reproducing r1 for code with reliable rewards. 2025. 5

[53] Xiaoyuan Liu, Tian Liang, Zhiwei He, Jiahao Xu, Wenxuan Wang, Pinjia He, Zhaopeng Tu, Haitao Mi, and Dong Yu. Trust, but verify: A self-verification approach to reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2505.13445*, 2025. 2

[54] Xiangyan Liu, Jinjie Ni, Zijian Wu, Chao Du, Longxu Dou, Haonan Wang, Tianyu Pang, and Michael Qizhe Shieh. Noisyrollout: Reinforcing visual reasoning with data augmentation. *arXiv preprint arXiv:2504.13055*, 2025. 5, 7

[55] Zichen Liu, Changyu Chen, Wenjun Li, Tianyu Pang, Chao Du, and Min Lin. There may not be aha moment in r1-zero-like training — a pilot study. https://oatllm.notion.site/oat-zero, 2025. Notion Blog. 4, 5

[56] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025. 7

[57] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. 6

[58] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02255*, 2023. 2, 5, 6

[59] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023. 2

[60] Brandon McKinzie, Zhe Gan, Jean-Philippe Fauconnier, Sam Dodge, Bowen Zhang, Philipp Dufter, Dhruti Shah, Xianzhi Du, Futang Peng, Floris Weers, et al. Mm1: Methods, analysis & insights from multimodal llm pre-training. *arXiv preprint arXiv:2403.09611*, 2024. 3

[61] Fanqing Meng, Lingxiao Du, Zongkai Liu, Zhixiang Zhou, Quanfeng Lu, Daocheng Fu, Tiancheng Han, Botian Shi, Wenhai Wang, Junjun He, et al. Mm-eureka: Exploring the frontiers of multimodal reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2503.07365*, 2025. 5, 6, 7

[62] Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. *arXiv preprint arXiv:2405.14734*, 2024. 2

[63] OpenAI. Openai o3-mini system card, 2025. https://cdn.openai.com/o3-mini-system-card-feb10.pdf, Last accessed on 2025-05-16. 6

[64] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022. 3

[65] Bhrij Patel, Souradip Chakraborty, Wesley A Suttle, Mengdi Wang, Amrit Singh Bedi, and Dinesh Manocha. Aime: Ai system optimization via multiple llm evaluators. *arXiv preprint arXiv:2410.03131*, 2024. 1, 4, 5, 6, 7

[66] Yingzhe Peng, Gongrui Zhang, Miaosen Zhang, Zhiyuan You, Jie Liu, Qipeng Zhu, Kai Yang, Xingzhong Xu, Xin Geng, and Xu Yang. Lmm-r1: Empowering 3b lmms with strong reasoning abilities through two-stage rule-based rl. *arXiv preprint arXiv:2503.07536*, 2025. 2, 4

[67] Runqi Qiao, Qiuna Tan, Guanting Dong, Minhui Wu, Chong Sun, Xiaoshuai Song, Zhuoma GongQue, Shanglin Lei, Zhe Wei, Miaoxuan Zhang, et al. We-math: Does your large multimodal model achieve human-like mathematical reasoning? *arXiv preprint arXiv:2407.01284*, 2024. 5, 6

[68] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020. 6

[69] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. 4

[70] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 2, 4, 5, 6, 7

[71] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023. 4, 2

[72] Core Team, Bingquan Xia, Bowen Shen, Cici, Dawei Zhu, Di Zhang, Gang Wang, Hailin Zhang, Huaqiu Liu, Jiebao Xiao, Jinhao Dong, Liang Zhao, Peidian Li, Peng Wang, Shihua Yu, Shimao Chen, Weikun Wang, Wenhan Ma, Xiangwei Deng, Yi Huang, Yifan Song, Zihan Jiang, Bowen Ye, Can Cai, Chenhong He, Dong Zhang, Duo Zhang, Guoan Wang, Hao Tian, Haochen Zhao, Heng Qu, Hongshen Xu, Jun Shi, Kainan Bao, QingKai Fang, Kang Zhou, Kangyang Zhou, Lei Li, Menghang Zhu, Nuo Chen, Qiantong Wang, Shaohui Liu, Shicheng Li, Shuhao Gu, Shuhuai Ren, Shuo Liu, Sirui Deng, Weiji Zhuang, Weiwei Lv, Wenyu Yang, Xin Zhang, Xing Yong, Xing Zhang,

Xingchen Song, Xinzhe Xu, Xu Wang, Yihan Yan, Yu Tu, Yuanyuan Tian, Yudong Wang, Yue Yu, Zhenru Lin, Zhichao Song, and Zihao Yue. Mimo: Unlocking the reasoning potential of language model – from pretraining to posttraining, 2025. 4, 7

[73] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. 6

[74] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, 2025. 4

[75] et al. Tianle Li*. From live data to high-quality benchmarks: The arena-hard pipeline, 2024. 6

[76] Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai Charitha Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, et al. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. *arXiv preprint arXiv:2406.16860*, 2024. 3

[77] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 3

[78] Ke Wang, Junting Pan, Weikang Shi, Zimu Lu, Mingjie Zhan, and Hongsheng Li. Measuring multimodal mathematical reasoning with math-vision dataset. *arXiv preprint arXiv:2402.14804*, 2024. 2, 5, 6

[79] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution, 2024. 6

[80] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. 4, 2

[81] Xiyao Wang, Zhengyuan Yang, Chao Feng, Hongjin Lu, Linjie Li, Chung-Ching Lin, Kevin Lin, Furong Huang, and Lijuan Wang. Sota with less: Mcts-guided sample selection for data-efficient visual reasoning self-improvement. *arXiv preprint arXiv:2504.07934*, 2025. 5

[82] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. 4

[83] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022. 3

[84] Yichen Wei, Yi Peng, Xiaokun Wang, Weijie Qiu, Wei Shen, Tianyidan Xie, Jiangbo Pei, Jianhao Zhang, Yunzhuo Hao, Xuchen Song, et al. Skywork r1v2: Multimodal hybrid reinforcement learning for reasoning. *arXiv preprint arXiv:2504.16656*, 2025. 7

[85] Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang, Zhenda Xie, Yu Wu, Kai Hu, Jiawei Wang, Yaofeng Sun, Yukun Li, Yishi Piao, Kang Guan, Aixin Liu, Xin Xie, Yuxiang You, Kai Dong, Xingkai Yu, Haowei Zhang, Liang Zhao, Yisong Wang, and Chong Ruan. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding, 2024. 6

[86] An Yang and et al Anfeng Li. Qwen3 technical report, 2025. 6, 7

[87] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. 5, 7

[88] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024. 4, 7

[89] Yi Yang, Xiaoxuan He, Hongkun Pan, Xiyan Jiang, Yan Deng, Xingtao Yang, Haoyu Lu, Dacheng Yin, Fengyun Rao, Minfeng Zhu, et al. R1-onevision: Advancing generalized multimodal reasoning through cross-modal formalization. *arXiv preprint arXiv:2503.10615*, 2025. 2, 5, 6, 7

[90] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023. 4

[91] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. Minicpm-v: A gpt-4v level mllm on your phone. *arXiv preprint arXiv:2408.01800*, 2024. 3

[92] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*, 2024. 3

[93] Lifan Yuan, Ganqu Cui, Hanbin Wang, Ning Ding, Xingyao Wang, Jia Deng, Boji Shan, Huimin Chen, Ruobing Xie, Yankai Lin, et al. Advancing llm reasoning generalists with preference trees. *arXiv preprint arXiv:2404.02078*, 2024. 5

[94] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *arXiv preprint arXiv:2311.16502*, 2023. 2

[95] Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, et al. Mmmu-pro: A more robust multi-discipline multimodal understanding benchmark. *arXiv preprint arXiv:2409.02813*, 2024. 2

[96] Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhu Chen. Acecoder: Acing

coder rl via automated test-case synthesis. *arXiv preprint arXiv:2502.01718*, 2025. 5

[97] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025. 4, 5, 7

[98] Jingyi Zhang, Jiaxing Huang, Huanjin Yao, Shunyu Liu, Xikun Zhang, Shijian Lu, and Dacheng Tao. R1-vl: Learning to reason with multimodal large language models via step-wise group relative policy optimization. *arXiv preprint arXiv:2503.12937*, 2025. 5

[99] Mozhi Zhang, Mianqiu Huang, Rundong Shi, Linsen Guo, Chong Peng, Peng Yan, Yaqian Zhou, and Xipeng Qiu. Calibrating the confidence of large language models by eliciting fidelity. *arXiv preprint arXiv:2404.02655*, 2024. 8

[100] Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Peng Gao, et al. Mathverse: Does your multi-modal llm truly see the diagrams in visual math problems? *arXiv preprint arXiv:2403.14624*, 2024. 5, 6

[101] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025. 5, 7

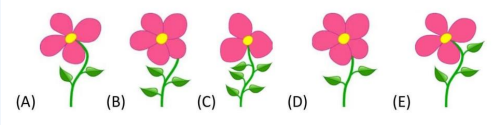
[102] Hengguang Zhou, Xirui Li, Ruochen Wang, Minhao Cheng, Tianyi Zhou, and Cho-Jui Hsieh. R1-zero's" aha moment" in visual reasoning on a 2b non-sft model. *arXiv preprint arXiv:2503.05132*, 2025. 2, 4

[103] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. 4

A. Full Responses for Multimodal Example

SYSTEM: You are a helpful AI Assistant that provides well-reasoned and detailed responses. You first think about the reasoning process as an internal monologue and then provide the user with the answer. Respond in the following format: <think> </think>\n<answer> </answer>

USER: The ladybird would like to sit on his flower. The flower has five petals and the stem has three leaves. On which flower should the ladybird sit?



ASSISTANT: <think> Let me inspect each flower carefully. Flower A appears to have exactly five petals, but its stem only sprouts two leaves. Flower B has a nice regular star of five petals—and I thought I spotted three little leaves tucked along its stem too. Flower C looks squashed, maybe only four broad petals and actually closer to four leaves as well. D has a perfect pentagon of petals but now that I look I remember only two leaves there. E seemed promising at first glance but I can't quite recall its leaf count. On balance, my initial gut is that D stands out with a clear five-petal bloom and I vaguely thought three leaves peeked out—but I must have misremembered. Still, I'll go with D for now. </think>
<answer>D</answer>

<think> **Re-checking D**, I see exactly two leaves—one on each side of the stem—so D definitely fails the "three leaves" criterion. The next candidate is E: I recall E had a single top-right leaf, another mid-left leaf, and one lower-left leaf, giving three in total, and its petals looked evenly spaced to make five. So E fits "five petals, three leaves," so my second guess will be E. </think>
<answer>E</answer>

<think> **Oops**—I've miscounted again. On closer inspection, E actually only has four fully visible petals because one is hidden behind the stem's curve. Meanwhile, B clearly displays five distinct petals and exactly three leaves along its stem (one low-left, one mid-right, one upper-left). Therefore the only flower meeting both criteria is B. </think>
<answer>B</answer>

Figure 7. Qualitative multimodal example of recursive think–answer process. The model refines its answer over multiple reasoning cycles by correcting initial counting errors and verifying object-level visual details, ultimately identifying the only flower that meets both criteria.

B. Algorithm of Recursive Think-Answer Process

Algorithm 1 Recursive Think-Answer Process for LLMs and VLMs

Require: Pre-trained Confidence Generator $\mathcal{C}_\phi$ and Pre-trained LLMs/VLMs $\pi_{\theta_{\text{init}}}$
Set reference model $\pi_{\text{ref}} \leftarrow \pi_{\theta_{\text{init}}}$
Set the training model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for sample a batch $\mathcal{B}$ in Dataset **do**
  Copy and freeze model $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
  Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot|q)$, until M maximum recursive.
  Extract the responses G' in G outputs until the early correct responses.
  Replace the responses: $G \leftarrow G'$
  Compute rewards and advantages for G outputs by confidence generator & answer parsing
  for Confidence generator Updating iteration $= 1, 2, \cdots, \mu$ **do**
    Update $\mathcal{C}_\phi$ by using Eq. (6)
  end for
  for LLMs/VLMs Updating iteration $= 1, 2, \cdots, \mu$ **do**
    Update π_θ by using Equation Eq. (3)
  end for
end for

C. Detailed Comparison between Performance and Computation Complexity

Comparison with Related Refinement and Verification Methods. We compare R-TAP against four closely related baselines: (a) Reflexion [71], (b) Self-Consistency [80], (c) Self-Refine [59], and (d) Self-Verification [53]. All methods are evaluated under the same output token budget. Unlike iterative refinement or verification-based approaches, which rely on explicit self-feedback loops, majority voting, or external verification during inference, R-TAP learns when to continue or terminate reasoning via reinforcement learning. As a result, it removes the need for repeated trial-and-error generation at deployment. Although the confidence generator in R-TAP is trained with binary supervision, it does not function as a hard 0/1 classifier at inference. Instead, it produces a continuous signal in $[0, 1]$, which enables the model to measure confidence improvement across recursive steps and regulate reasoning depth smoothly.

	MaxTokenLimit	AIME25	HMMT Feb25	OmniMath	GPQA	LCB	Avg	Oops-style count	Decoding token count	Training Time
Phi-4-reasoning-plus	32768	78.0	53.6	81.9	69.3	65.9	69.7	15.7	14509.7	-
w/ Reflexion [71]	32768	80.2	56.1	83.4	71.3	66.5	71.5	18.9	15230.4	51 hours
w/ Self-Consistency [80]	32768	81.1	55.8	84.0	72.2	66.4	71.9	16.8	16012.9	58 hours
w/ Self-Refine [59]	32768	78.5	54.9	83.1	71.0	68.5	71.2	14.3	13890.6	45 hours
w/ Self-Verification [53]	32768	82.4	57.6	85.7	73.5	64.8	72.8	17.5	14890.3	51 hours
R-TAP	32768	**83.7**	**60.3**	**86.2**	**76.7**	**72.1**	**75.8**	**5.6**	**4378.8**	**28 hours**

	MaxTokenLimit	MMMU	MathVista	OlympiadBench	MathVision	MMMU-Pro	Avg	Oops-style count	Output token count	Training Time
R1V2-38B	12000	73.6	74.0	62.6	49.0	52.0	62.2	17.2	9890.7	-
w/ Reflexion [71]	12000	74.5	74.8	63.0	49.5	53.7	63.1	17.5	10989.3	57 hours
w/ Self-Consistency [80]	12000	75.0	76.0	64.0	50.0	52.5	63.5	18.6	10678.5	60 hours
w/ Self-Refine [59]	12000	73.8	74.2	62.0	49.0	55.5	62.9	15.3	9938.2	59 hours
w/ Self-Verification [53]	12000	76.5	77.0	65.0	51.0	53.0	64.5	17.8	10183.8	58 hours
R-TAP	12000	**78.2**	**82.3**	**69.4**	**56.8**	**59.2**	**69.2**	**8.5**	**5789.4**	**39 hours**

Token Efficiency and Computational Cost. R-TAP does not increase total output tokens. On the contrary, it substantially reduces them. While recursive sampling is introduced during training, the learned policy discourages unnecessary self-corrections at inference time. As shown in this table, R-TAP reduces output tokens by approximately 2–3$\times$ compared to self-consistency, self-refine, and verification-based baselines under the same output token budget, while achieving higher accuracy.

T	G	AIME25	HMMT Feb25	OmniMath	GPQA	LiveCodeBench	Avg	Output token count	Training Time
1	4	76.1	52.1	80.4	67.9	64.4	68.2	15034.9	51 hours
1	8	77.2	52.9	81.2	68.6	65.2	69.0	15210.7	52 hours
1	12	78.0	53.6	81.9	69.3	65.9	69.7	14872.5	58 hours
2	12	80.9	57.1	84.1	73.3	68.6	72.6	9258.9	42 hours
3	12	82.3	58.9	85.0	75.1	70.3	74.9	6127.4	34 hours
4	12	83.7	60.3	86.2	76.7	72.1	75.8	4378.8	28 hours
4	8	83.0	59.7	85.7	76.0	71.4	75.2	4442.3	25 hours
4	4	82.2	58.9	85.0	75.2	70.6	74.4	4321.6	22 hours

T	G	MMMU	MathVista	OlympiadBench	MathVision	MMMU-Pro	Avg	Decoding token count	Training Time
1	4	72.4	72.6	61.2	47.8	50.6	60.9	10342.7	55 hours
1	8	73.1	73.4	62.0	48.5	51.4	61.7	9927.3	62 hours
1	12	73.6	74.0	62.6	49.0	52.0	62.2	10168.9	68 hours
2	12	76.1	77.0	65.3	52.7	55.0	64.1	8234.6	55 hours
3	12	77.5	79.4	67.2	54.9	57.8	67.4	6912.1	46 hours
4	12	78.2	82.3	69.4	56.8	59.2	69.2	5789.4	39 hours
4	8	77.8	81.7	68.8	56.4	58.7	68.7	5698.2	34 hours
4	4	77.1	80.8	67.9	55.6	57.8	67.8	5861.5	30 hours

Effect of Majority Voting. We further evaluate self-consistency with varying voting numbers (N), under identical output token budgets (32768 for Phi-4-reasoning-plus and 12000 for R1V2-38B). As expected, increasing N consistently improves performance across all models. However, R-TAP-trained models already achieve strong performance with $N = 1$, and additional voting yields only marginal gains. This indicates that R-TAP does not replace self-consistency; rather, it learns a more stable single-sample reasoning policy, thereby reducing reliance on majority voting during inference.

Voting N	AIME25	HMMT Feb25	OmniMath	GPQA	LiveCodeBench	Avg
1	83.7	60.3	86.2	76.7	72.1	75.8
3	83.9	60.5	86.4	76.9	72.3	76.0
5	84.0	60.6	86.5	77.0	72.4	76.1
7	84.1	60.7	86.6	77.1	72.5	76.2
9	84.1	60.7	86.6	77.1	72.5	76.2
12	84.2	60.8	86.7	77.2	72.6	76.3
15	84.2	60.8	86.7	77.2	72.6	76.3
18	84.2	60.8	86.7	77.2	72.6	76.3

Voting N	MMMU	MathVista	OlympiadBench	MathVision	MMMU-Pro	Avg
1	78.2	82.3	69.4	56.8	59.2	69.2
3	78.3	82.4	69.5	56.9	59.3	69.3
5	78.3	82.4	69.5	56.9	59.3	69.3
7	78.4	82.5	69.6	57	59.4	69.4
9	78.5	82.6	69.7	57.1	59.5	69.5
12	78.5	82.6	69.7	57.1	59.5	69.5
15	78.5	82.6	69.7	57.1	59.5	69.5
18	78.5	82.6	69.7	57.1	59.5	69.5

Recap. R-TAP consists of four key components: (a) GRPO-based reinforcement learning for Think-Answer trajectories, (b) a recursive reward mechanism derived from both the confidence generator and intermediate results, (c) suppression of unnecessary Oops-style refinement, and (d) improved inference efficiency resulting from reduced refinement steps.

Importantly, while prior uncertainty- or refinement-based approaches use confidence signals for reranking, filtering, or verification after generation, R-TAP integrates confidence as an internal reinforcement signal during training. This signal continuously modulates recursive reasoning depth, directly shaping the learned reasoning policy instead of performing post-hoc correction at inference time.